Area Science Park
Research and Technology Institute
Laboratory of Data Engineering

Web Application for DPCfam and DPCstruct Data Exploration

Report 7 (24 Feb - 2 Mar 2026) : Web Development (DPCStruct Backend)

Emmanuel Nyandu Kagarabi
Master in Data Management and Curation (MDMC - SISSA)

Supervisors :
Valerio Piomponi
Elaheh Saadat

February 28, 2026

1 Introduction

This report is the seventh in a sequence documenting the weekly assigned tasks of our internship. Focusing on the DPCStruct dataset (Barone et al., 2025), and building upon Report 6, We present the initial dpcstruct model (`DpcStructMcsProperty`), which allows users to explore properties of dpcstruct metaclusters and compare them with Pfam-36 matching domains (Clans or families, if any). We have configured the project to be multi-pushed (on my side) to both personal and RitAreaSciencePark repositories. From now on, any feedback and/or modifications can be easily made in the Rit repository, after which I will pull changes, modify/update them, and push them to all repositories at once. All CSV data are available here.

2 Development

2.1 Database Creation in PostgreSQL

We developed a notebook (check it out here) aimed at preprocessing and merging relevant features from `mcs_properties.tsv` and `dpcstruct_consistency.tsv` files, resulting in a consistent dataframe (Figure 1):

	mc_id	mc_size	len_aa	len_std	len_ratio	plddt	disorder	tmscore	lddt	pident	pfam_score	pfam_labels
0	MC0	43	112.02	12.36	0.11	78.53	0.29	0.54	0.65	23.74	NaN	NONE
1	MC1	19	68.89	7.43	0.11	90.81	0.28	0.77	0.82	25.87	NaN	NONE
2	MC2	19	368.05	52.78	0.14	85.00	0.30	0.46	0.56	19.60	NaN	NONE
3	MC3	18	346.39	34.27	0.10	82.09	0.25	0.55	0.49	14.00	NaN	NONE
4	MC4	12	135.17	13.06	0.10	86.24	0.22	0.69	0.62	23.25	NaN	NONE
5	MC5	46	159.04	38.84	0.24	81.34	0.19	0.56	0.57	17.34	1.00	CL0263
6	MC6	20	242.60	19.41	0.08	71.32	0.33	0.62	0.70	24.50	1.00	PF14802
7	MC7	6	79.00	3.96	0.05	74.06	0.18	0.84	0.82	57.20	1.00	PF00110
8	MC8	24	85.71	6.73	0.08	79.14	0.41	0.71	0.82	49.28	1.00	CL0465
9	MC9	7	50.57	4.81	0.10	81.26	0.19	0.85	0.88	82.23	0.75	CL0272-CL0016
10	MC10	16	62.06	4.97	0.08	81.09	0.22	0.89	0.87	46.48	1.00	CL0186

Figure 1: DPCStruct Metaclusters Properties - Comparison with PFam.

Returning to our routine, we created a table in PostgreSQL (Figure 2) with fields matching `dpcstruct_mcs_properties.csv` (Figure 1):

```
-- Function: Stores the biological and structural properties of protein metaclusters and their consistency with Pfam-36 labels.

CREATE TABLE IF NOT EXISTS dpcstruct_mcs_properties (
    mc_id VARCHAR(50) PRIMARY KEY,
    mc_size INTEGER,
    len_aa DOUBLE PRECISION,
    len_std DOUBLE PRECISION,
    len_ratio DOUBLE PRECISION,
    plddt DOUBLE PRECISION,
    disorder DOUBLE PRECISION,
    tmscore DOUBLE PRECISION,
    lddt DOUBLE PRECISION,
    pident DOUBLE PRECISION,
    pfam_score DOUBLE PRECISION,
    pfam_labels TEXT
);


-- Natural Numeric Sorting: Extracts numbers from 'MC123' for fast integer-based sorting.
-- Used to sort metaclusters numerically (MC1, MC2, MC10) instead of alphabetically.
CREATE INDEX IF NOT EXISTS idx_dpcstruct_mcs_prop_num ON dpcstruct_mcs_properties (CAST(SUBSTRING(mc_id FROM '[0-9]+') AS INTEGER));
```

Figure 2: PostgreSQL `dpcstruct_mcs_properties` table.

The table was then populated with data from the CSV file (Figure 3):

```
static > scripts > dpcstruct > populate_dpcstruct_tables.sql > ...
    PGSQL Disconnected
1   -- ================================================================================
2   -- I. DATA POPULATION (\copy commands)
3   -- ================================================================================
4   -- After creating tables, we populate them with data, by executing the following commands in psql in the given order (Parent tables first):
5   \copy dpcstruct_mcs_properties FROM 'static/dataframes/dpcstruct/dpcstruct_mcs_properties.csv' WITH (FORMAT csv, HEADER true);
```

Figure 3: Populating PostgreSQL `dpcstruct_mcs_properties` table.

2.2 Django Routine

We updated Report 3, following the **Model→Filter→Table→View→Template** pattern:

2.2.1 Model : dpcstruct/models.py

The Model represents our PostgreSQL table (Figure 2) in Django:

```
dpcstruct > models.py > DpcStructMcsProperty
1   from django.db import models
2   class DpcStructMcsProperty(models.Model):
3       # A. MC properties
4       mc_id = models.CharField(max_length=50, primary_key=True)
5       mc_size = models.IntegerField(null=True, blank=True)
6       len_aa = models.FloatField(null=True, blank=True)
7       len_std = models.FloatField(null=True, blank=True)
8       len_ratio = models.FloatField(null=True, blank=True)
9       plddt = models.FloatField(null=True, blank=True)
10      disorder = models.FloatField(null=True, blank=True)
11      tmscore = models.FloatField(null=True, blank=True)
12      lddt = models.FloatField(null=True, blank=True)
13      pident = models.FloatField(null=True, blank=True)
14      # B. PFAM-related fields
15      pfam_score = models.FloatField(null=True, blank=True)
16      pfam_labels = models.TextField(null=True, blank=True)
17      class Meta:
18          db_table = 'dpcstruct_mcs_properties'
19          managed = False
20          verbose_name = 'DPCStruct MC Property'
21          verbose_name_plural = 'DPCStruct MC Properties'
22
23      def __str__(self):
24          return self.mc_id
```

Figure 4: DpcStructMcsProperty Model : Each field in the model corresponds to a column in our PostgreSQL table (Figure 2), we do not allow django to create/alter/delete it via migrations since it already exists(`managed=False`), MCID is used as the primary key (no need to autoincrement `id`), and when django displays an instance, it shows the MCID. We display a desired(verbose) name for the admin panel.

2.2.2 Filter : dpcstruct/views.py

Filtering enables users to search for specific metaclusters:

```python
import django_filters
from django import forms
from .models import DpcStructMcsProperty

class DpcStructMcsPropertyFilter(django_filters.FilterSet):
    mc_id = django_filters.CharFilter(
        lookup_expr="icontains",
        widget=forms.TextInput(attrs={
            'placeholder': 'Enter MCID',
            'class': 'form-control'
        })
    )

    class Meta:
        model = DpcStructMcsProperty
        fields = ["mc_id"]
```

Figure 5: DpcStructMcsPropertyFilter : Enter a MCID(case-insensitive), we display all MCIDs which contain ID, via a form to call a view with `?mc_id=MCID` in the URL.

2.2.3 Tables : dpcstruct/tables.py

This module defines which fields are displayed and how they are rendered.

```python
import django_tables2 as tables
from .models import DpcStructMcsProperty

class DpcStructMcsPropertyTable(tables.Table):
    mc_id = tables.LinkColumn("dpcstruct_detail", args=[tables.A("mc_id")], verbose_name="MCID")
    mc_size = tables.Column(verbose_name="Size")
    len_aa = tables.Column(verbose_name="Avg. Len.")
    len_std = tables.Column(verbose_name="Len. Std")
    len_ratio = tables.Column(verbose_name="Len. Ratio")
    plddt = tables.Column(verbose_name="Plddt")
    disorder = tables.Column(verbose_name="Disorder")
    tmscore = tables.Column(verbose_name="TM-Score")
    lddt = tables.Column(verbose_name="Lddt")
    pident = tables.Column(verbose_name="Pident")
    pfam_score = tables.Column(verbose_name="Score Pfam", empty_values=())
    pfam_labels = tables.Column(verbose_name="Pfam Labels")

    def render_pfam_score(self, value):
        if value is None:
            return "N/A"
        # Check for NaN if it's a float
        import math
        if isinstance(value, float) and math.isnan(value):
            return "N/A"
        return value

    class Meta:
        model = DpcStructMcsProperty
        template_name = "django_tables2/bootstrap.html"
        attrs = {"class": "table table-striped table-hover table-bordered"}
        fields = (
            'mc_id',
            'mc_size',
            'len_aa',
            'len_std',
            'len_ratio',
            'plddt',
            'disorder',
            'tmscore',
            'lddt',
            'pident',
            'pfam_score',
            'pfam_labels',
        )
```

Figure 6: DpcStructMcsPropertyTable: We specify which model fields to display as table columns, we then use `django_tables2` library to autogenerate an HTML table which is styled using Bootstrap template.

2.2.4 Views : dpcstruct/views.py

Views integrate data retrieval, filtering, pagination, and rendering:

```python
from django_tables2.views import SingleTableMixin
from django_filters.views import FilterView
from .models import DpcStructMcsProperty
from .tables import DpcStructMcsPropertyTable
from .filters import DpcStructMcsPropertyFilter

class DpcStructPropertyListView(SingleTableMixin, FilterView):
    model = DpcStructMcsProperty
    table_class = DpcStructMcsPropertyTable
    filterset_class = DpcStructMcsPropertyFilter
    paginate_by = 10
    template_name = 'dpcstruct/dpcstruct_list_metaclusters.html'

    def get_queryset(self):
        # We sort by numeric part of MCID
        return DpcStructMcsProperty.objects.extra(
            select={'mc_num': "CAST(SUBSTRING(mc_id FROM '[0-9]+') AS INTEGER)"}
        ).order_by('mc_num')
```

Figure 7: DpcStructPropertyListView : The rendered HTML table (DpcStructMcsPropertyTable) and filter form (DpcStructMcsPropertyFilter) are combined and paginated (10 rows per page), then passed to the template, MCIDs are sorted based on IDs values.

2.2.5 URLs : dpcstruct/urls.py

URLs route HTTP requests to appropriate views. We have three routes, all pointing to the same view for now:

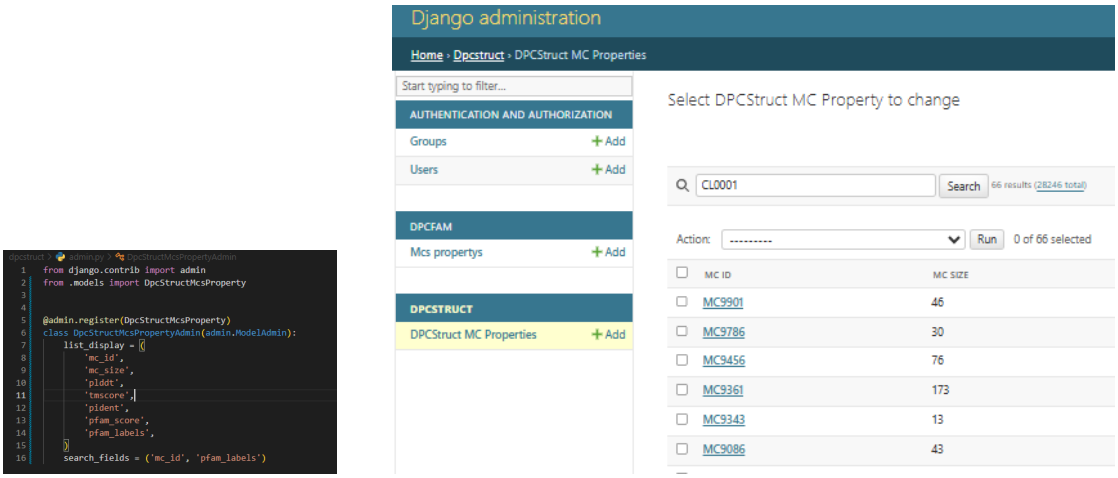
```python
from django.urls import path
from .views import DpcStructPropertyListView

urlpatterns = [
    path('', DpcStructPropertyListView.as_view(), name='index'),
    path('mcs/', DpcStructPropertyListView.as_view(), name='dpcstruct_mcs_list'),
    # Placeholder for detail view
    path('dpcstruct/<str:mc_id>/', DpcStructPropertyListView.as_view(), name='dpcstruct_detail'),
]
```

Figure 8: DPCStruct URLs : When a user visits "/" or "/mcs/", Django calls `DpcStructPropertyListView`, processes the GET request, and renders the template.

2.2.6 Admin : dpcstruct/admin.py

The admin interface allows staff users to manage data:



(a) Admin Registry (b) Admin interface

Figure 9: DPCStruct admin interface : Staff users (created using `python3 manage.py createsuperuser`)) can log in at "/admin/" to view, filter, search (by `mc_id` or `Pfam_label`), and edit metacluster properties.

2.2.7 Template : dpcstruct/dpcstruct_list_metaclusters.html

The HTML template renders the page that the user sees in their browser:

```
{% extends "index.html" %}
{% load render_table from django_tables2 %}

{% block title %}DPCStruct | Metaclusters{% endblock %}

{% block content %}
<style>
  /* Hide django-tables2 default pagination inside the table wrapper */
  .django-tables2 > tfoot,
  .table-wrapper .pagination,
  .table-wrapper nav {
    display: none !important;
  }
  /* Style for placeholder text */
  input::placeholder {
    color: #999;
    font-style: italic;
  }
</style>

<div class="container mt-4">
  <h1 class="mb-3">DPCStruct Metaclusters</h1>
```

Figure 10: DPCStruct template : We master the DRY principle. Basically, the filter form submits query parameters via the URL (?mcid=<value>), and Django renders a paginated table with proper structure and styling.

2.2.8 Final Result: Paginated DPCSTRUCT Metaclusters Table

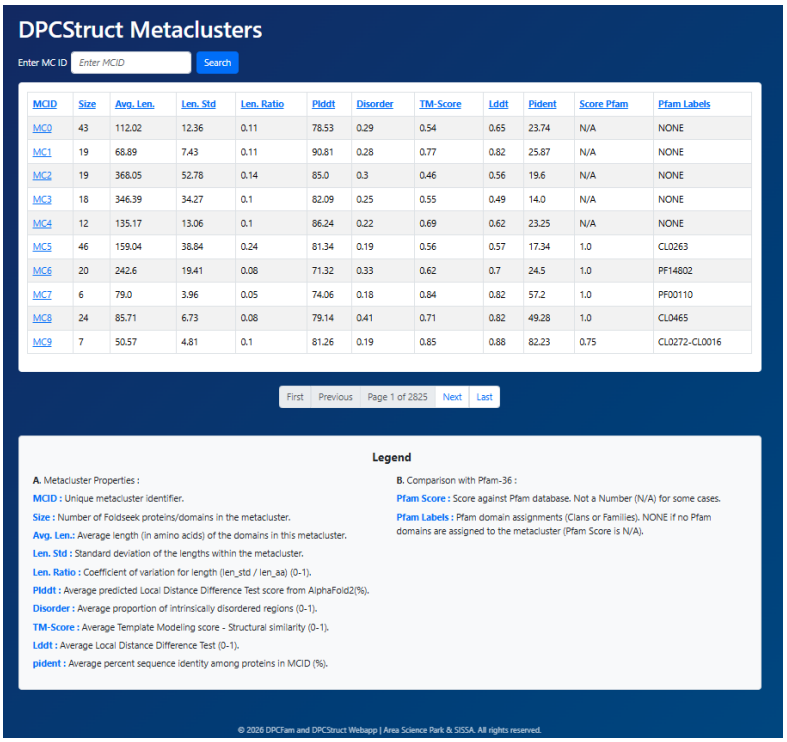


Figure 11: DPCStruct Metaclusters page : After running `python3 manage.py (makemigrations , migrate , runserver)`, we may visit the page at http://localhost:8000/dpcstruct/.

2.3 Reproducibility

We have updated the README.md file for reproducing the current state of the project. Check it out for reproducibility; dpcstruct instructions are also integrated (**4.2.II**).

3 Conclusion

To display this draft version, a local PostgreSQL database is required. We will shortly extend the application by introducing additional tables and establishing relationships among them, as discussed in Report 6.

References

Barone, F., Laio, A., Punta, M., Cozzini, S., Ansuini, A., and Cazzaniga, A. (2025). Unsupervised domain classification of alphafold2-predicted protein structures. *PRX Life*, 3(2):023009.